The Journal of Open Source Software

poLCAParallel: A C++ reimplementation and parallelisation of poLCA

Sherman Edea Lo[1]

1 Queen Mary, University of London, UK

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 27 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

The R package poLCAParallel is a fork and reimplementation of an existing R package, poLCA. In this fork, the underlying algorithm was parallelised and rewritten in C++. This offers a significant speedup, making it suitable for large datasets using high-performance computing (HPC). The API was kept the same as the original code to allow poLCAParallel to be a seamless drop-in replacement of poLCA.

Statement of need

The R (R Core Team, 2025) package poLCA (Linzer & Lewis, 2011, 2026) stands for polytomous variable latent class analysis. This studies a mixture model of random multivariate polytomous variables (Bandeen-Roche et al., 1997; Goodman, 1974) and fits the model to observed data.

Let Y be the multivariate polytomous random variable of interest with K independent categories. This can be represented as a random vector $Y = (Y_1, Y_2, \cdots, Y_K)$. Each category can take one of many independent outcomes with values $Y_k \in \{1, 2, \cdots, L_k\}$ for $k = 1, 2, \cdots, K$ where L_k is the number of outcomes for the kth category. The outcomes do not necessarily need to have an ordering.

For the latent class model, with the number of classes J and a prior probability for each class π_j for $j = 1, 2, \cdots, J$ where $0 < \pi_j < 1$ and $\sum_{j=1}^{J} \pi_j = 1$, the probability mass function is given as

$$\mathbb{P}(Y = y) = \sum_{j=1}^{J} \pi_j \prod_{k=1}^{K} \prod_{l=1}^{L_k} p_{j,k,l}^{\delta(y_k, l)}$$

where $y = (y_1, y_2, \cdots, y_K)$ is a possible instance of Y, $p_{j,k,l}$ is the outcome probability for class j, category k and outcome l, and $\delta(\cdot, \cdot)$ is the Kronecker delta. The outcome probabilities are such that $0 < p_{j,k,l} < 1$ for all j, k and l denoted above. They are also normalised such that $\sum_{l=1}^{L_k} p_{j,k,l} = 1$ for all j and k denoted above.

Suppose a dataset of n observations of Y was obtained, each observation denoted as $y^{(1)}, y^{(2)}, \cdots, y^{(n)}$. The log-likelihood is given as

$$\ln L = \sum_{i=1}^{n} \ln \left[\mathbb{P}\left(Y = y^{(i)}\right) \right]$$

For a given dataset, the expectation–maximisation (EM) algorithm (Dempster et al., 1977) estimates the parameters of the model $p_{j,k,l}$ and π_j for all j, k and l as denoted previously. This is done by maximising the log-likelihood using an iterative procedure, alternating between taking an E step and an M step.

In the E step, the posterior probabilities for each observation belonging to different classes, given the model parameters, are estimated. In the M step, the values of $p_{j,k,l}$ and π_j are

DRAFT

updated to maximise the log-likelihood given those posterior probabilities. The E and M steps are repeated until some convergence criteria are met. The algorithm is known to converge to a local maximum log-likelihood. Thus, different initial values should be trialled to find solutions that maximise the log-likelihood globally.

The package poLCA also offers models where π_j is a linear function of observed predictor vectors (Bandeen-Roche et al., 1997). The package can also calculate the standard errors (Bandeen-Roche et al., 1997; Cramér, 1999; Rao et al., 1992) and goodness-of-fit (Pearson, 1900; Wilks, 1938) statistics.

The need for optimising poLCA was identified by an HPC user at Queen Mary, University of London (QMUL). The HPC at QMUL is called Apocrita (King et al., 2017) and offers CPU and GPU compute facilities. It was identified that a user's job using poLCA took many days and exceeded the 10-day maximum runtime.

State of the field

Within the R ecosystem, poLCA (Linzer & Lewis, 2011, 2026) is heavily used by statisticians and in data analysis, such as in Schimmenti (2018), Tromp et al. (2018), Godfred-Cato et al. (2020), Whitaker et al. (2022), Romiti et al. (2024) and Xiao et al. (2024). The majority of the package is written in R, with a few small functions in C.

The package poLCA showed some significant bottlenecks when our user used it for a dataset consisting of about half a million observations. The code in the package was designed to be run in serial. Critically, it was found that inefficient operations were used which can bottleneck the package. For example, in the standard error calculations, the pseudoinverse (Barata & Hussein, 2012; Penrose, 1955) was used to directly invert a matrix in a linear problem involving symmetric matrices. In the goodness-of-fit calculations, a sorting algorithm was used to find unique observations of Y in the dataset, rather than using tools specifically for finding unique items. These inefficiencies caused the user's code to appear stalled during the 10-day runtime.

We contributed to poLCA by implementing our optimisation as a fork. The optimisations would change the internal code completely and poLCAParallel could feasibly stand alone as its own package. This would avoid overwhelming the original code repository and authors with large code suggestions and pull requests.

Software design

The underlying EM algorithm was written in C++ (ISO, 2020) and parallelised by trialing out different initial values in parallel. Hence, the package was named poLCAParallel.

It was designed to be a drop-in replacement for poLCA, keeping the same API as the original package with a few additional optional parameters and functions. This allows regular users of poLCA to adopt poLCAParallel by only changing a few lines of code.

The C++ code uses Armadillo (Sanderson & Curtin, 2025, 2019) for linear algebra, which offers faster speeds than an implementation in R (Eddelbuettel & Sanderson, 2014). The C++ component of the code was written so that it can be compiled on its own outside the R ecosystem when needed. The R package RcppArmadillo (Eddelbuettel, Francois, Bates, et al., 2026; Eddelbuettel & Sanderson, 2014) was used so that the C++ implementation can be compiled with Armadillo and run from R using Rcpp (Eddelbuettel, 2013; Eddelbuettel, Francois, Allaire, et al., 2026; Eddelbuettel & Balamuta, 2018; Eddelbuettel & Francois, 2011).

To improve cache efficiency (Sehr & Andrist, 2018), some matrices and vectors from R were reordered. For example, in the log-likelihood calculation, which involves iterating $p_{j,k,l}$ over j, k and l, the outcome probabilities were reordered such that neighbouring elements in memory are accessed during iterations, improving performance.

DRAFT

In terms of parallelisation, the class `std::thread` from the standard library `<thread>` was used (ISO, 2020; Sehr & Andrist, 2018). Each thread uses different initial values for the EM algorithm in parallel. By using more initial values, the solution with the highest log-likelihood is more likely to achieve a global maximum. And by running from these initial values in parallel, such a solution can be calculated faster.

Another good candidate tool for multi-thread programming is OpenMP (Dagum & Menon, 1998) because it provides programmers a way to write thread-safe code without having to use mutexes. However, using the C++ standard library was decided instead because it was more familiar to the author at the time.

The standard error calculations and goodness-of-fit tests were also reimplemented in C++. Many of the linear algebra operations were rewritten to avoid direct or pseudo-inversion of a matrix. Because covariance matrices are symmetric, this property was used to replace direct or pseudo-inversion of matrices with appropriate Armadillo linear solvers and decompositions. This offers better numerical stability and faster speeds (Kiusalaas, 2014).

The goodness-of-fit statistics require counts of all unique observations of Y. In the original code, the response variables were sorted and then counted individually. The package poLCAParallel offers significant speed-up by using a hash map (Sehr & Andrist, 2018) to keep track of and look up all unique instances of Y.

Documentation

The documentation in the R code and C++ code can be generated using the R package roxygen2 (Wickham, Danenberg, et al., 2026) and the software doxygen (Van Heesch & others, 2026), respectively. The C++ documentation is mainly aimed at future developers and maintainers.

Testing

The C++ component was designed to be isolated from the R ecosystem, and can be compiled and tested by itself using Catch2 (Hořeňovský et al., 2026). The package testthat (Wickham, 2011; Wickham, Posit Software, et al., 2026) was used to test the entire package through the R interface. This allows the testing to be done at different levels, pinpointing problems, should one occur, in the pipeline, whether it's in the R or C++ component.

Because poLCAParallel is a drop-in replacement for poLCA, tests include ensuring the output of poLCA and poLCAParallel are the same or numerically similar. For example, objects should have the same property names and content. However, changes to the linear algebra operations and improvements to the numerical stability may mean some standard error results will diverge from the original code and cannot be compared for similarity in tests.

Package availability

The package is available in a GitHub repository and from CRAN (Lo et al., 2026). The package poLCAParallel uses roxygen2 and RcppArmadillo, which generate documentation and code for it to work in R. To avoid committing and tracking changes of generated code, the default main branch does not include any generated code. A branch, called package, was created, which includes all generated code so that the package can be used from R straight from that branch. A caveat is that if a user wishes to install poLCAParallel directly via GitHub in R, they will have to specify the branch name, such as `remotes::install_github("QMUL/poLCAParallel@package")`.

DRAFT

Research impact statement

The Apocrita user, whose code was bottlenecked by poLCA, managed to run their code using poLCAParallel in the order of hours rather than days. In Eto et al. (2023), they acknowledged poLCAParallel and the author.

Apocrita's main CPU compute facility consists of two Intel Xeon Platinum 8268 CPUs per node. A job can use up to 48 cores per node. A benchmark was conducted using the user's data. The data is confidential but it has the following summary statistics:

- $n = 489\,478$
- $K = 204$
- $L_k = 2$ for all k

A model with $J = 10$ classes was fitted, using 48 initial values. By default, poLCA calculates the standard errors and goodness-of-fit statistics in addition to the model fit, so these will be included in the benchmarks.

To allow for poLCA to run in a feasible time, a random subsample of $n = 10\,000$ was taken and used in the benchmarks. They are compared with poLCAParallel using different numbers of threads, as shown in Figure 1. A speedup of at least 6× was observed in the single-thread performance when using poLCAParallel over poLCA, with further improvements when using more threads.

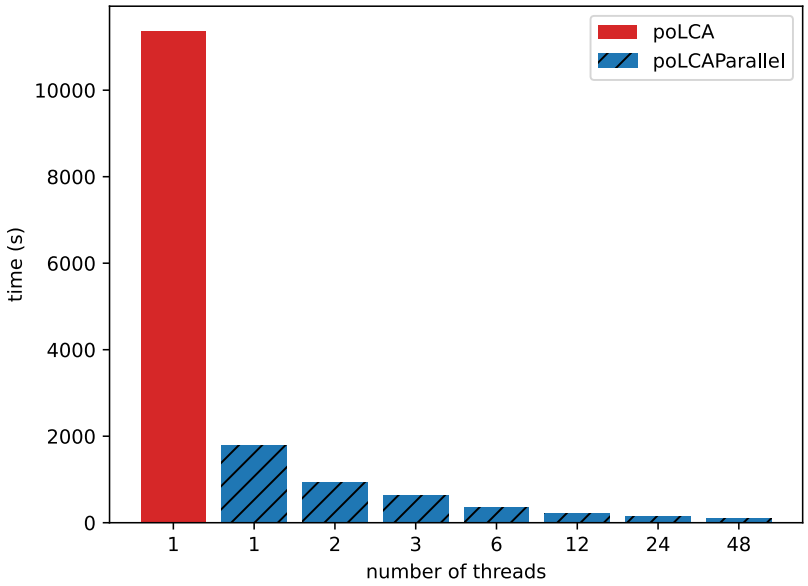


Figure 1: Benchmarks comparing poLCA with poLCAParallel fitting a model with $J = 10$ classes onto a random subsample $n = 10\,000$ of the data. For poLCAParallel, different numbers of threads were used.

For the entire dataset of $n = 489\,478$, poLCA would take days to complete, thus omitted in further benchmarks. Figure 2 shows the benchmarks of poLCAParallel when fitting the model onto the entire dataset. The entire fit was completed in the order of hours rather than days when using poLCAParallel. Whilst using more threads does improve performance, diminishing returns were observed when the entire node of 48 cores was used. In this realm, it is bottlenecked by single-thread calculations such as standard errors and goodness-of-fit statistics.

DRAFT

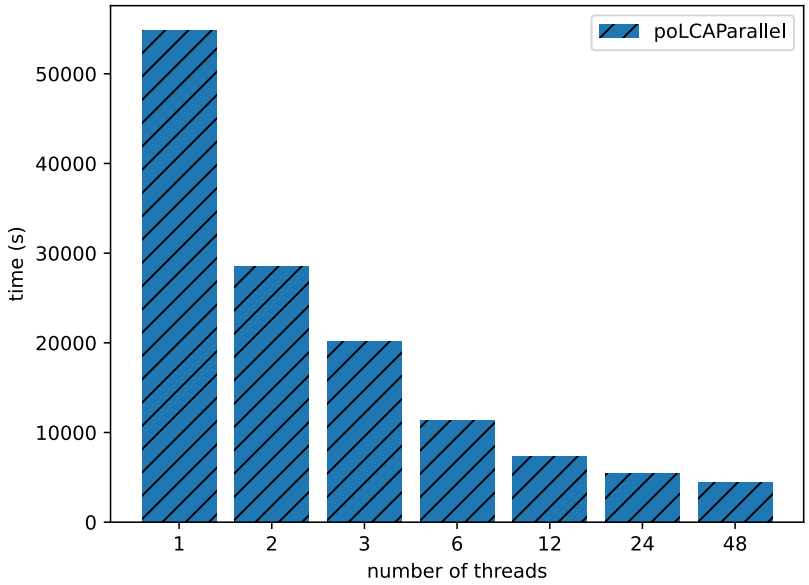


Figure 2: Benchmark of poLCAParallel fitting a model with $J = 10$ classes onto the entire dataset. Different numbers of threads were used.

During development of poLCAParallel, blog posts were written about the process on our departmental website, such as Lo (2022) and Lo (2026). The former was cited in Alvarez-Galvez et al. (2023), Zhou et al. (2023), Priya et al. (2026). Otherwise, the package was mentioned in Kassa-Sombo et al. (2026) and Zhao et al. (2026). This shows that the package poLCAParallel was reached out to and used by researchers who were also bottlenecked by poLCA.

AI usage disclosure

No artificial intelligence tools were used in the creation of this paper or the R package poLCAParallel.

Alvarez-Galvez, J., Ortega-Martin, E., Ramos-Fiol, B., Suarez-Lledo, V., & Carretero-Bravo, J. (2023). Epidemiology, mortality, and health service use of local-level multimorbidity patterns in South Spain. *Nature Communications*, *14*(1), 7689–7616. https://doi.org/10.1038/s41467-023-43569-5

Bandeen-Roche, K., Miglioretti, D. L., Zeger, S. L., & Rathouz, P. J. (1997). Latent variable regression for multiple discrete outcomes. *Journal of the American Statistical Association*, *92*(440), 1375–1386. https://doi.org/10.2307/2965407

Barata, J. C. A., & Hussein, M. S. (2012). The Moore–Penrose pseudoinverse: A tutorial review of the theory. *Brazilian Journal of Physics*, *42*(1-2), 146–165. https://doi.org/10.1007/s13538-011-0052-z

Cramér, H. (1999). *Mathematical methods of statistics* (Vol. 9). Princeton University Press.

Dagum, L., & Menon, R. (1998). OpenMP: An industry standard API for shared-memory programming. *IEEE Computational Science and Engineering*, *5*(1), 46–55. https://doi.org/10.1109/99.660313

Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, *39*(1), 1–22. https://doi.org/10.1111/j.2517-6161.1977.tb01600.x

DRAFT

Eddelbuettel, D. (2013). *Seamless R and C++ integration with Rcpp*. Springer. https://doi.org/10.1007/978-1-4614-6868-4

Eddelbuettel, D., & Balamuta, J. J. (2018). Extending R with C++: A brief introduction to Rcpp. *The American Statistician*, *72*(1), 28–36. https://doi.org/10.1080/00031305.2017.1375990

Eddelbuettel, D., & Francois, R. (2011). Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, *40*(8), 1–18. https://doi.org/10.18637/jss.v040.i08

Eddelbuettel, D., Francois, R., Allaire, J., Ushey, K., Kou, Q., Russell, N., Ucar, I., Bates, D., & Chambers, J. (2026). *Rcpp: Seamless R and C++ integration*. https://doi.org/10.32614/CRAN.package.Rcpp

Eddelbuettel, D., Francois, R., Bates, D., Ni, B., & Sanderson, C. (2026). *RcppArmadillo: 'Rcpp' integration for the 'Armadillo' templated linear algebra library*. https://doi.org/10.32614/CRAN.package.RcppArmadillo

Eddelbuettel, D., & Sanderson, C. (2014). RcppArmadillo: Accelerating R with high-performance C++ linear algebra. *Computational Statistics & Data Analysis*, *71*, 1054–1063. https://doi.org/10.1016/j.csda.2013.02.005

Eto, F., Samuel, M., Henkin, R., Mahesh, M., Ahmad, T., Angdembe, A., Hamish McAllister-Williams, R., Missier, P., J. Reynolds, N., R. Barnes, M., Hull, S., Finer, S., & Mathur, R. (2023). Ethnic differences in early onset multimorbidity and associations with health service use, long-term prescribing, years of life lost, and mortality: A cross-sectional study using clustering in the UK clinical practice research datalink. *PLOS Medicine*, *20*(10), 1–23. https://doi.org/10.1371/journal.pmed.1004300

Godfred-Cato, S., Bryant, B., Leung, J., Oster, M. E., Conklin, L., Abrams, J., Roguski, K., Wallace, B., Prezzato, E., Koumans, E. H., Lee, E. H., Geevarughese, A., Lash, M. K., Reilly, K. H., Pulver, W. P., Thomas, D., Feder, K. A., Hsu, K. K., Plipat, N., … Belay, E. (2020). COVID-19–associated multisystem inflammatory syndrome in children — United States, March–July 2020. *MMWR. Morbidity and Mortality Weekly Report*, *69*(32), 1074–1080. https://doi.org/10.15585/mmwr.mm6932e2

Goodman, L. A. (1974). Exploratory latent structure analysis using both identifiable and unidentifiable models. *Biometrika*, *61*(2), 215–231. https://doi.org/10.1093/biomet/61.2.215

Hořeňovský, M., Nash, P., Thrasher, C., Macrae, C., Grajciar, J., Moene, M., Kosta, PureAbstract, Jeřábek, M., offa, Vertexwahn, Baruch, Huebl, A., coombez, dvirtz, NeroBurner, Seifert, D., Mijoski, D., Pfiffikus, … Franzen, S. (2026). *Catchorg/Catch2: v3.15.3* (Version v3.15.3). Zenodo. https://doi.org/10.5281/zenodo.21611439

ISO. (2020). *ISO/IEC 14882:2020: Programming languages — C++* (Sixth edition). International Organization for Standardization. https://www.iso.org/standard/79358.html

Kassa-Sombo, A., Migeon, A., Godillon, L., Laurent, E., Grammatico-Guillon, L., & Guillon, A. (2026). Characterizing ICU-like profiles of very old patients hospitalized in medical intermediate care units in France: A clustering analysis of a nationwide population-based study. *Annals of Intensive Care*, *16*, 100031. https://doi.org/10.1016/j.aicoj.2026.100031

King, T., Butcher, S., & Zalewski, L. (2017). *Apocrita - high performance computing cluster for Queen Mary University of London*. https://doi.org/10.5281/zenodo.438045

Kiusalaas, J. (2014). *Numerical methods in engineering with Python 3* (Third edition). Cambridge University Press. https://doi.org/10.1017/CBO9781139523899

Linzer, D. A., & Lewis, J. B. (2011). poLCA: An R package for polytomous variable latent class analysis. *Journal of Statistical Software*, *42*(10), 1–29. https://doi.org/10.18637/jss.v042.i10

DRAFT

Linzer, D. A., & Lewis, J. B. (2026). *poLCA: Polytomous variable latent class analysis*. https://doi.org/10.32614/CRAN.package.poLCA

Lo, S. E. (2022). *Speeding up and parallelising R packages (using Rcpp and C++) - QMUL ITS research blog*. https://blog.hpc.qmul.ac.uk/speeding_up_r_packages/.

Lo, S. E. (2026). *Modernising Rcpp and C++ code with std::span - QMUL ITS research blog*. https://blog.hpc.qmul.ac.uk/cpp-span/.

Lo, S. E., Linzer, D., & Lewis, J. (2026). *poLCAParallel: Polytomous variable latent class analysis parallel*. https://doi.org/10.32614/CRAN.package.poLCAParallel

Pearson, K. (1900). X. On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, *50*(302), 157–175. https://doi.org/10.1080/14786440009463897

Penrose, R. (1955). A generalized inverse for matrices. *Mathematical Proceedings of the Cambridge Philosophical Society*, *51*(3), 406–413. https://doi.org/10.1017/S0305004100030401

Priya, T. S., Yan, H., Wangensteen, K. J., Wu, S., Luehrs, A. C., Pinto e Vairo, F., Leng, F., Acosta, A. J., Vierkant, R. A., Allen, A. M., Lazaridis, K. N., Klee, E. W., Yao, D. D., & Tian, S. (2026). Subtyping metabolic dysfunction-associated steatotic liver disease using electronic health record-linked genomic cohorts reveals diverse etiologies and progression. *Nature Communications*, *17*(1), 10428–10428. https://doi.org/10.1038/s41467-026-77410-6

R Core Team. (2025). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. https://doi.org/10.32614/R.manuals

Rao, C. R., Kotz, S., & Johnson, N. L. (1992). Information and the accuracy attainable in the estimation of statistical parameters. In *Breakthroughs in statistics* (pp. 235–247). Springer New York. https://doi.org/10.1007/978-1-4612-0919-5_16

Romiti, G. F., Corica, B., Mei, D. A., Bisson, A., Boriani, G., Olshansky, B., Chao, T.-F., Huisman, M. V., Proietti, M., & Lip, G. Y. H. (2024). Patterns of comorbidities in patients with atrial fibrillation and impact on management and long-term prognosis: An analysis from the prospective global GLORIA-AF registry. *BMC Medicine*, *22*(1), 151–112. https://doi.org/10.1186/s12916-024-03373-4

Sanderson, C., & Curtin, R. (2025). Armadillo: An efficient framework for numerical linear algebra. *2025 17th International Conference on Computer and Automation Engineering (ICCAE)*, 303–307. https://doi.org/10.1109/ICCAE64891.2025.10980539

Sanderson, C., & Curtin, R. (2019). Practical sparse matrices in C++ with hybrid storage and template-based expression optimisation. *Mathematical and Computational Applications*, *24*(3). https://doi.org/10.3390/mca24030070

Schimmenti, A. (2018). The trauma factor: Examining the relationships among different types of trauma, dissociation, and psychopathology. *Journal of Trauma & Dissociation*, *19*(5), 552–571. https://doi.org/10.1080/15299732.2017.1402400

Sehr, V., & Andrist, B. (2018). *C++ high performance: Boost and optimize the performance of your C++ 17 code* (First edition). Packt Publishing.

Tromp, J., Tay, W. T., Ouwerkerk, W., Teng, T.-H. K., Yap, J., MacDonald, M. R., Leineweber, K., McMurray, J. J. V., Zile, M. R., Anand, I. S., Lam, C. S. P., & authors, A.-H. (2018). Multimorbidity in patients with heart failure from 11 Asian regions: A prospective cohort study using the ASIAN-HF registry. *PLOS Medicine*, *15*(3), 1–22. https://doi.org/10.1371/journal.pmed.1002541

DRAFT

Van Heesch, D., & others. (2026). *Doxygen*. https://www.doxygen.nl/

Whitaker, M., Elliott, J., Chadeau-Hyam, M., Riley, S., Darzi, A., Cooke, G., Ward, H., & Elliott, P. (2022). Persistent COVID-19 symptoms in a community study of 606,434 people in England. *Nature Communications*, *13*(1), 1957–1910. https://doi.org/10.1038/s41467-022-29521-z

Wickham, H. (2011). Testthat: Get started with testing. *The R Journal*, *3*, 5–10. https://doi.org/10.32614/RJ-2011-002

Wickham, H., Danenberg, P., Csárdi, G., Eugster, M., & Posit Software. (2026). *roxygen2: In-line documentation for R*. https://doi.org/10.32614/CRAN.package.roxygen2

Wickham, H., Posit Software, P., & team, R. C. (2026). *Testthat: Unit testing for R*. https://doi.org/10.32614/CRAN.package.testthat

Wilks, S. S. (1938). The Large-Sample Distribution of the Likelihood Ratio for Testing Composite Hypotheses. *The Annals of Mathematical Statistics*, *9*(1), 60–62. https://doi.org/10.1214/aoms/1177732360

Xiao, Y., Bi, K., Yip, P. S.-F., Cerel, J., Brown, T. T., Peng, Y., Pathak, J., & Mann, J. J. (2024). Decoding suicide decedent profiles and signs of suicidal intent using latent class analysis. *JAMA Psychiatry*, *81*(6), 595–605. https://doi.org/10.1001/jamapsychiatry.2024.0171

Zhao, S., Cheng, J., Wang, H., & Wu, Q. (2026). Latent class analysis–derived classification and cancer-specific death in endometrial carcinoma: A population-based cohort study. *JCO Precision Oncology*, *10*(10), e2600393. https://doi.org/10.1200/PO-26-00393

Zhou, W., He, M.-M., Wang, F., Xu, R.-H., Wang, F., & Zhao, Q. (2023). Latent class analysis-derived classification improves the cancer-specific death stratification of molecular subtyping in colorectal cancer. *NPJ Precision Oncology*, *7*(1), 60–11. https://doi.org/10.1038/s41698-023-00412-w

DRAFT